

Lab 4 - VibeCoding: Hangman Challenge

Chat History (Tasks 1-4)

Prompts used for each task, with the LLM's answer in short.

Task 1 - Guess-state correctness

Prompt

I'm working on a Python Hangman game (game.py attached).

Bug: in the guess() method, guessing the same WRONG letter twice costs a life each time, because only self.guessed (correct letters) is checked, not self.wrong.

Task: Fix it so any previously attempted letter, correct or wrong, is rejected with "Already guessed." and changes no state (no life lost, no change to the sets).

Constraints:

- Make a minimal change, don't rewrite the file.
- Keep everything in memory, no new dependencies.
- Explain what you changed and why.

Answer (short)

The duplicate check now covers both sets, so a repeated wrong letter is rejected before any state changes.

```
if letter in self.guessed or letter in self.wrong:
    return "Already guessed."
```

Task 2 - Session statistics

Prompt

Task 2: integrate the SessionStats class from stats.py into HangmanGame in game.py.

Requirements:

- Create one SessionStats in HangmanGame.__init__ (not in start_round), so it persists across rounds.
- At the end of each completed round (win or loss), call stats.record(won, streak).
- start_round() must reset only round-specific state (secret, guessed, wrong, lives, hint_used), not the stats, score or streak.
- When the session ends (including via /quit or "n" to another round), print rounds played, rounds won, and best streak.

Constraints: minimal changes, in-memory only, no new dependencies.

Explain what you changed.

Answer (short)

Imported SessionStats and created self.stats once in __init__. Called self.stats.record(True, self.streak) after the streak is increased on a win, and self.stats.record(False, self.streak) after it is reset on a loss. A round ended with /quit is not recorded. Added print_session_stats() and called it on every way the session can end. start_round() was left unchanged.

Task 3 - Difficulty and scoring

Prompt

Task 3: add difficulty choices to the Hangman game (game.py).

Requirements:

- Add easy, medium, hard that change starting lives and scoring, e.g. a

```
dict of settings (lives, win points, hint penalty).
- Player chooses difficulty at the start of each round, after
category selection. Keep category selection working as before.
- start_round() applies the chosen difficulty's lives, so state does not
leak between rounds.
- Hint usage must affect scoring consistently using that difficulty's
penalty (still once per round, score never below 0).
- Reject an invalid difficulty with a message and ask again, without
changing state.
```

Constraints: minimal changes, in-memory only, no new dependencies.
Explain what you changed.

Answer (short)

Added a DIFFICULTIES dictionary, self.difficulty in __init__, lives from the chosen difficulty in start_round(), the hint penalty in use_hint(), and win points in play_round(). run() asks for a difficulty after the category and repeats the question on invalid input.

```
DIFFICULTIES = {
    "easy": {"lives": 8, "win_points": 3, "hint_penalty": 1},
    "medium": {"lives": 6, "win_points": 5, "hint_penalty": 1},
    "hard": {"lives": 4, "win_points": 8, "hint_penalty": 2},
}
```

Follow-up: the pasted code had an indentation error in play_round(); the LLM was asked for the corrected file and the code was fixed.

Task 4 - Robust input and feedback

Prompt

Task 4: improve input handling in game.py without rewriting the project.

Requirements:

- Invalid letters (digits, symbols, multiple characters, empty input) must never change game state, and should print one clear message.
- Unknown commands starting with "/" (e.g. "/foo") should print "Unknown command." and change nothing.
- Repeated /hint should print "Hint already used." once, with no extra score penalty.
- Category and difficulty prompts should accept "q" to quit cleanly (showing session stats), and invalid choices should print one message and ask again.
- Each player action should produce exactly one feedback message, not duplicated.

Constraints: minimal changes, in-memory only, no new dependencies, keep code modular. Explain what you changed.

Answer (short)

play_round() now handles /quit, /hint (hint text or 'Hint already used.') and any other command starting with '/' ('Unknown command.'). Everything else goes to guess(), which rejects invalid or repeated letters before changing state. The category and difficulty prompts accept q and print the session statistics before exiting.

```
if raw == "/quit":
    return False
if raw == "/hint":
    hint = self.use_hint()
    print("Hint already used." if hint is None else hint)
    continue
if raw.startswith("/"):
    print("Unknown command.")
    continue
print(self.guess(raw))
```

Testing done

- Before: repeated wrong letter (z, z, z) lost a life each time (6 -> 5 -> 4 -> 3). Recorded in before.mp4.
- After: repeated wrong letter prints 'Already guessed.' and lives stay the same. - Hard difficulty starts with 4 lives, easy with 8; lives do not carry over between rounds. - Invalid input (digits, 'ab', empty), 'foo', repeated '/hint' and '/quit' leave the game state unchanged. - Session statistics (rounds played, rounds won, best streak) print when the session ends. Recorded in after.mp4.